

SISTEMA DE CATÁLOGO DE FILMES

Documentação do Projeto – Programação Orientada a Objetos

Integrantes do Grupo

Eduardo Niquini Santiago
Fábio Sérgio Carvalho Mendonça
Guilherme Cândido Vidulino
Heitor Freitas Fernandes
Lucas Marinho Blon Rocha
Matheus Souto dos Santos

1. Apresentação do Sistema

O Sistema de Catálogo de Filmes tem como objetivo permitir o gerenciamento de filmes, séries, avaliações e comentários realizados pelos usuários. O projeto foi modelado em C# utilizando os princípios da Programação Orientada a Objetos (POO), visando modularidade, reutilização de código e facilidade de manutenção.

O sistema CineLog, desenvolvido em C# com .NET 8, oferece as seguintes funcionalidades completas:

- Cadastro e autenticação de usuários (com hash de senha).
- Cadastro de filmes e séries com atributos específicos (duração/diretor para filmes; temporadas/episódios/criador para séries)
- Avaliação de mídias com notas de 0 a 10 e comentários
- Três listas personalizadas por usuário: "Quero assistir", "Assistidos" e "Favoritos"
- Filtros por gênero, ano e nota mínima
- Busca textual por título
- Top 10 mídias mais bem avaliadas
- Recomendações automáticas baseadas nos gêneros favoritos do usuário.
- Persistência em JSON (repositórios) com escrita atômica e sincronização em tempo real entre múltiplos clientes
- Interface via Console totalmente colorida e interativa
- Tratamento completo de exceções (domínio próprio)
- O sistema conta com uma API REST ([ASP.NET](#) Core) que expõe endpoints para todas as operações

Além disso, uma versão web completa (HTML/CSS/JS) foi desenvolvida, comunicando-se via API REST com o backend C# ([ASP.NET](#) Core), compartilhando o mesmo modelo de dados e regras de negócio em tempo real.

2. Modelagem do Sistema

Descrição de todas as classes do sistema, com seus papéis, atributos, métodos e como estão interligadas:

Camada de Modelos (Entidades de Domínio)

Classe Abstrata Midia

Papel: Representar qualquer conteúdo do catálogo (filme ou série), centralizando atributos e comportamentos comuns. É abstrata porque não existe uma "mídia genérica" no domínio – apenas filmes ou séries.

Atributos principais:

- int Id – identificador único
- string Titulo – nome da obra

- int AnoLancamento – ano de lançamento original
- string Genero – gênero principal (ex: "Drama", "Ficção Científica")
- string Sinopse – breve descrição da trama
- List<Avaliacao> _avaliacoes – lista privada de avaliações
- double NotaMedia (derivada) – média aritmética das avaliações

Métodos principais:

- void AdicionarAvaliacao(Avaliacao a) – insere nova avaliação e recalcula a média
- bool PodeAvaliar(int usuariold) – verifica se o usuário já avaliou
- abstract string ObterTipo() – retorna "Filme" ou "Série" (polimorfismo)
- abstract string ObterDetalhes() – retorna informações específicas (polimorfismo)

Encapsulamento relevante: NotaMedia possui private set, sendo modificada apenas internamente pelo método AdicionarAvaliacao. A lista _avaliacoes é exposta como IReadOnlyList<Avaliacao>.

Classe Filme (herda de Midia)

Papel: Especializar Midia com atributos exclusivos de filmes.

Atributos adicionais:

- int DuracaoMinutos – duração em minutos
- string Diretor – nome do diretor principal

Métodos sobrescritos:

- override string ObterTipo() → retorna "Filme"
- override string ObterDetalhes() → exibe diretor e duração

Validações: diretor não pode ser vazio; duração deve ser maior que zero.

Classe Serie (herda de Midia)

Papel: Especializar Midia com atributos exclusivos de séries.

Atributos adicionais:

- int NumeroTemporadas – total de temporadas lançadas
- int NumeroEpisodios – total de episódios lançados
- bool EmAndamento – indica se ainda está em produção
- string Criador – nome do criador da série

Métodos sobrescritos:

- override string ObterTipo() → retorna "Série"
- override string ObterDetalhes() → exibe criador, temporadas, episódios e status

Validações: temporadas e episódios devem ser maiores que zero; criador não pode ser vazio.

Classe Usuario

Papel: Representar um usuário do sistema, armazenando suas listas pessoais e credenciais.

Atributos:

- int Id – identificador único
- string Nome – nome completo
- string Email – e-mail único (utilizado para login)
- string _senhaHash – hash da senha (nunca armazenada em texto puro)
- List<int> _queroAssistir – IDs das mídias que o usuário deseja ver
- List<int> _assistidos – IDs das mídias já assistidas
- List<int> _favoritos – IDs das mídias favoritas

Propriedades expostas (somente leitura):

- IReadOnlyList<int> QueroAssistir
- IReadOnlyList<int> Assistidos
- IReadOnlyList<int> Favoritos

Métodos principais:

- bool ValidarSenha(string senha) – verifica a senha fornecida contra o hash armazenado
- void AdicionarNaLista(TipoLista lista, int midiaId) – adiciona uma mídia à lista especificada
- void RemoverDaLista(TipoLista lista, int midiaId) – remove da lista especificada

Regra de negócio: ao adicionar uma mídia em "Assistidos", ela é automaticamente removida de "Quero Assistir".

Encapsulamento: a senha é armazenada como hash e nunca exposta publicamente. As listas internas são manipuladas apenas pelos métodos públicos, garantindo consistência.

Classe Avaliacao

Papel: Representar o vínculo único entre um usuário e uma mídia, armazenando a nota e o comentário.

Atributos:

- int Id – identificador único
- int Usuarioid – referência ao usuário que avaliou
- int MidiaId – referência à mídia avaliada
- double Nota – valor entre 0 e 10
- string Comentario – texto opcional do usuário
- DateTime DataAvaliacao – data e hora da avaliação

Validações: a nota deve estar entre 0 e 10 (inclusive). Valores inválidos lançam DadosInvalidosException.

Imutabilidade: após criada, uma avaliação não pode ser alterada (apenas recriada).

Enum TipoLista

Papel: Definir os três tipos de lista que cada usuário pode possuir.

Exemplo: `public enum TipoLista { QueroAssistir, Assistidos, Favoritos }`

Camada de Interfaces (Contratos

IMidiaRepositorio

Define as operações de persistência para mídias. Permite trocar a implementação (JSON, banco de dados, etc.) sem alterar as regras de negócio.

Métodos:

- void Adicionar(Midia midia)
- Midia ObterPorId(int id)
- IEnumerable<Midia> ObterTodos()
- IEnumerable<Midia> FiltrarPorGenero(string genero)
- IEnumerable<Midia> FiltrarPorAno(int ano)
- IEnumerable<Midia> FiltrarPorNotaMinima(double notaMinima)
- void Salvar() / void Carregar()
- int Proximoid()

IUsuarioRepositorio

Define as operações de persistência para usuários.

Métodos:

- void Adicionar(Usuario usuario)
- Usuario ObterPorId(int id)
- Usuario ObterPorEmail(string email)
- IEnumerable<Usuario> ObterTodos()
- bool EmailExiste(string email)
- void Salvar() / void Carregar()
- int Proximoid()

IAvaliacaoRepositorio

Define as operações de persistência para avaliações.

Métodos:

- void Adicionar(Avaliacao avaliacao)
- IEnumerable<Avaliacao> ObterPorMidia(int midiaId)
- IEnumerable<Avaliacao> ObterPorUsuario(int usuarioid)
- void Salvar() / void Carregar()
- int Proximoid()

Camada de Repositórios (Implementação)

MidiaRepositorioJson

Implementa IMidiaRepositorio com persistência em arquivos JSON na pasta data/midias.json. Utiliza classes DTO internas para serialização, pois as classes de domínio (Filme, Serie) possuem herança, o que dificulta a serialização direta.

Características:

- Salva automaticamente após cada modificação
- Suporta tanto filmes quanto séries no mesmo arquivo
- Restaura o estado completo (incluindo avaliações) ao carregar

UsuarioRepositorioJson

Implementa IUsuarioRepositorio com persistência em data/usuarios.json. Armazena o hash da senha e as listas pessoais de cada usuário.

Características:

- Utiliza reflexão para serializar o campo privado `_senhaHash`
- Restaura corretamente o hash ao carregar os dados

AvaliacaoRepositorioJson

Implementa `IAvaliacaoRepositorio` com persistência em `data/avaliacoes.json`.

Camada de Serviço (Lógica de Negócio)

CatalogoService

Papel: Concentrar **toda** a lógica de negócio do sistema. É a única camada que conhece e orquestra os repositórios. A UI (Console) nunca acessa repositórios diretamente.

Dependências (injetadas via construtor):

- `IMidiaRepositorio` `_midias`
- `IUsuarioRepositorio` `_usuarios`
- `IAvaliacaoRepositorio` `_avaliacoes`

Métodos principais (agrupados por responsabilidade):

Mídias:

- `Filme` `CadastrarFilme(...)`
- `Serie` `CadastrarSerie(...)`
- `Midia` `ObterMidia(int id)`
- `IEnumerable<Midia>` `ListarMidias()`
- `IEnumerable<Midia>` `FiltrarMidias(string? genero, int? ano, double? notaMinima)`

Usuários:

- `Usuario` `CadastrarUsuario(string nome, string email, string senha)`
- `Usuario` `Login(string email, string senha)`

Avaliações:

- `Avaliacao` `AvaliarMidia(int usuariold, int midiald, double nota, string comentario)`
- `IEnumerable<Avaliacao>` `ObterAvaliacoesDaMidia(int midiald)`

Listas:

- `void` `AdicionarNaLista(int usuariold, int midiald, TipoLista lista)`
- `void` `RemoverDaLista(int usuariold, int midiald, TipoLista lista)`

- IEnumerable<Midia> ObterListaDoUsuario(int usuarioid, TipoLista lista)

Recomendação e rankings:

- IEnumerable<Midia> RecomendarPorGenero(int usuarioid, int quantidade = 5)
- IEnumerable<Midia> ObterTopMidias(int quantidade = 10)

Camada de Exceções (Hierarquia Própria)

Todas as exceções do domínio herdam da classe base CineLogException, permitindo tratamento específico:

Exceção	Quando ocorre
DadosInvalidosException	Dados inválidos na criação de entidades (ex: nota fora de 0-10, título vazio).
AvaliacaoDuplicadaException	Usuário tenta avaliar a mesma mídia duas vezes.
EntidadeNaoEncontradaException	Mídia, usuário ou avaliação não encontrada pelo ID.
AutenticacaoException	E-mail ou senha incorretos no login.
EmailDuplicadoException	Tentativa de cadastrar e-mail já existente.

Camada de Interface com o Usuário

ConsoleUI

Papel: Fornecer uma interface interativa via terminal, colorida e amigável, utilizando o padrão **Facade** para simplificar o acesso ao CatalogoService.

Principais menus:

- Menu principal (login/criar conta/explorar)
- Menu do usuário logado (catálogo, listas, cadastrar mídia, top, recomendações)
- Navegação e exibição detalhada de mídias
- Formulários para cadastro de filmes e séries

Tratamento de exceções: captura exceções do domínio e exibe mensagens amigáveis ao usuário.

2.1. Arquitetura do Sistema

O projeto adota uma **arquitetura em camadas** clara, separando responsabilidades:

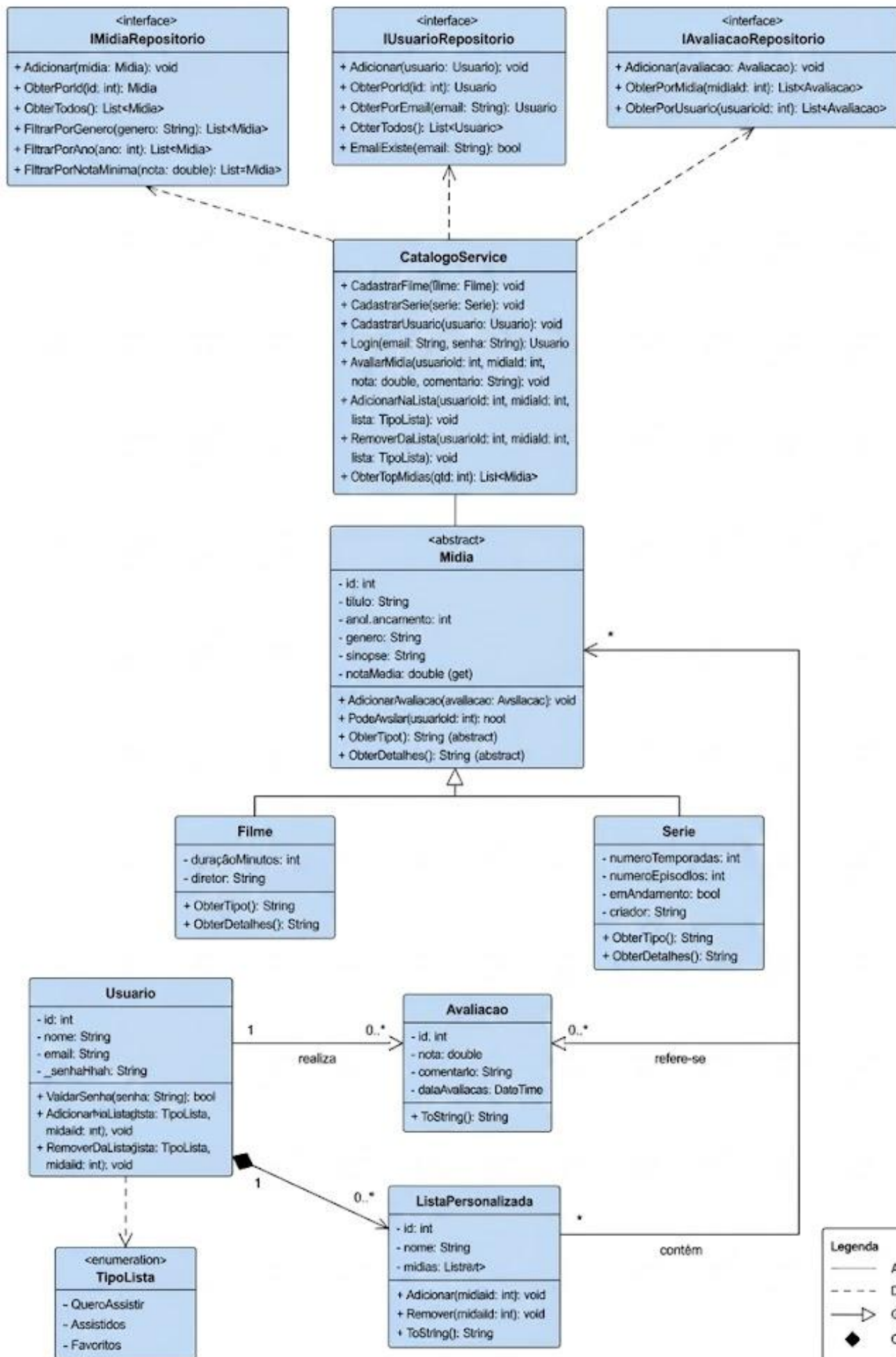
Camada	Pasta/Namespace	Responsabilidade
Models	CineLog.Models	Entidades de domínio (Midia, Filme, Serie, Usuario, Avaliacao)
DTOs	CineLog.DTOs	Contratos de entrada/saída da API (isolam o domínio)

Interfaces	CineLog.Interfaces	Contratos para repositórios (IMidiaRepositorio, IUsuarioRepositorio, IAvaliacaoRepositorio)
Repositories	CineLog.Repositories	Implementação concreta com persistência em JSON
Services	CineLog.Services	Lógica de negócio (CatalogoService)
Controllers	CineLog.Controllers	Endpoints da API REST (AuthController, MidiasController)
wwwroot	wwwroot/	Frontend HTML/CSS/JS servido pela própria API
Exceptions	CineLog.Exceptions	Hierarquia de exceções do domínio

2.2. Fluxo de Dados

1. O usuário interage com o frontend (index.html) no navegador.
2. O JavaScript faz requisições fetch() para a API REST (http://localhost:5000/api/...).
3. Os Controllers (AuthController, MidiasController) recebem as requisições HTTP.
4. Os Controllers chamam o CatalogoService (Service Layer).
5. CatalogoService aplica as regras de negócio e usa os repositórios.
6. Os repositórios leem/gravam arquivos JSON na pasta data/ (com escrita atômica).
7. A resposta JSON retorna ao frontend, que atualiza a interface.

2.3. Diagrama UML do Sistema



3. Aplicação dos Quatro Pilares da POO

A seguir, apresentamos como cada um dos quatro pilares da Programação Orientada a Objetos está concretamente aplicado no Sistema de Catálogo de Filmes, com exemplos reais das classes e métodos do projeto em sua versão final.

Abstração

A abstração consiste em modelar apenas o que é essencial para o domínio do problema, ocultando detalhes irrelevantes. No sistema, ela aparece em dois lugares principais:

Classe abstrata Midia

Representa o conceito genérico de "conteúdo disponível no catálogo". Nunca é instanciada diretamente — não existe um objeto "Midia" no mundo real — mas concentra tudo que Filme e Serie têm em comum: Id, Titulo, Ano, Genero, Sinopse, a lista de avaliacoes e o cálculo automático da MediaNota. Isso elimina duplicação de código nas subclasses.

Código real:

```
public abstract class Midia
{
    public int Id { get; protected set; }
    public string Titulo { get; protected set; }
    public double NotaMedia => _avaliacoes.Any() ?
Math.Round(_avaliacoes.Average(a => a.Nota), 1) : 0;

    public abstract string ObterTipo();
    public abstract string ObterDetalhes();
}
```

Interfaces de repositório

As interfaces IMidiaRepositorio, IUsuarioRepositorio e IAvaliacaoRepositorio definem contratos para a camada de persistência. Quem as utiliza (CatalogoService) não precisa saber se os dados vêm de JSON, banco de dados ou qualquer outra fonte — apenas conhece as operações disponíveis.

Encapsulamento

Encapsulamento é o princípio de proteger o estado interno dos objetos, expondo apenas o necessário por meio de interfaces controladas. O sistema aplica isso em várias situações:

MediaNota em Midia

O atributo MediaNota (calculado) é exposto apenas com get público. Seu valor é atualizado internamente pelo método AdicionarAvaliacao, que chama o cálculo da média. Nenhum código externo pode modificar a nota diretamente.

```
public double NotaMedia { get; private set; } // apenas leitura externa

public void AdicionarAvaliacao(Avaliacao avaliacao)
{
    _avaliacoes.Add(avaliacao);
    RecalcularMedia(); // método privado
}
```

Senha do Usuario

O campo _senhaHash é privado. Nenhuma outra classe acessa ou exibe esse valor diretamente — apenas o método ValidarSenha() verifica a senha fornecida contra o hash armazenado.

Listas pessoais do Usuario

As listas _queroAssistir, _assistidos e _favoritos são campos privados. O acesso externo é feito apenas por meio de propriedades somente-leitura (IReadOnlyList<int>) e métodos públicos controlados (AdicionarNaLista, RemoverDaLista), garantindo que nenhum código externo corrompa as listas.

```
private readonly List<int> _queroAssistir = new();
public IReadOnlyList<int> QueroAssistir => _queroAssistir.AsReadOnly();
```

Herança

A herança permite que classes derivadas reutilizem atributos e comportamentos de uma classe-pai, especializando apenas o que é diferente. O sistema possui uma hierarquia clara:

Classe-pai	Classe-filha	O que é herdado	O que é especializado
Midia (abstract)	Filme	Id, Titulo, Ano, Genero, Sinopse, avaliacoes, MediaNota, AdicionarAvaliacao	+ DuracaoMinutos, + Diretor. Implementação de ObterTipo() e ObterDetalhes()
Midia	Serie	Todos os atributos e	+

(abstract)		métodos de Midia	NumeroTemporadas , + NumeroEpisodios, + EmAndamento, + Criador. Implementação de ObterTipo() e ObterDetalhes()
------------	--	------------------	---

Com essa estrutura, Filme e Serie não repetem código para gerenciar avaliações ou armazenar dados básicos — herdam tudo isso de Midia e acrescentam apenas o que os distingue.

Código real:

```
public class Filme : Midia
{
    public int DuracaoMinutos { get; private set; }
    public string Diretor { get; private set; }

    public override string ObterTipo() => "Filme";
    public override string ObterDetalhes() => $"Diretor: {Diretor} | Duração: {DuracaoMinutos} min";
}

public class Serie : Midia
{
    public int NumeroTemporadas { get; private set; }
    public int NumeroEpisodios { get; private set; }
    public bool EmAndamento { get; private set; }
    public string Criador { get; private set; }

    public override string ObterTipo() => "Série";
    public override string ObterDetalhes() => $"Criador: {Criador} | Temporadas: {NumeroTemporadas} | Episódios: {NumeroEpisodios}";
}
```

Polimorfismo

Polimorfismo permite que um mesmo método se comporte de formas diferentes dependendo do tipo real do objeto. No sistema, isso é central para o funcionamento do catálogo.

Sobrescrita de ObterTipo() e ObterDetalhes()

Os métodos abstratos declarados em Midia são implementados de forma diferente em Filme e Serie. O CatalogoService pode trabalhar com uma lista de Midia e chamar esses métodos sem saber se cada objeto é um Filme ou uma Serie — cada um responde corretamente.

Código real:

```
// O serviço trata tudo como Midia
IEnumerable<Midia> midias = _midias.ObterTodos();

foreach (Midia m in midias)
{
    Console.WriteLine(m.ObterTipo());      // "Filme" ou "Série"
    Console.WriteLine(m.ObterDetalhes());  // detalhes específicos de cada
tipo
}
```

Saída para um filme: "Filme" e "Diretor: Christopher Nolan | Duração: 148 min"

Saída para uma série: "Série" e "Criador: Vince Gilligan | Temporadas: 5 | Episódios:62"

Por que isso importa?

O CatalogoService opera sobre List<Midia> ou IEnumerable<Midia> sem precisar de estruturas if/else para distinguir Filme de Serie. Se no futuro o sistema ganhar uma nova classe (ex: Documentario, Anime, Podcast) que herde de Midia, nenhum código existente no Service ou na UI precisará ser alterado — o polimorfismo

Pilar	Onde aparece no código	Por que foi essa escolha
Abstração	Midia (classe abstrata), interfaces I...Repositorio	Evita instanciar conceitos genéricos; impõe contratos para as subclasses e para a persistência
Encapsulamento	MediaNota (private set), _senhaHash, listas como IReadOnlyList	Protege integridade dos dados; controla acesso externo
Herança	Filme : Midia, Serie : Midia	Reutiliza código da classe-pai; espelha a realidade do domínio (filme e série são tipos de mídia)
Polimorfismo	ObterTipo() e ObterDetalhes() em Filme, Serie	CatalogoService opera sem if/else; facilita extensão futura

4. Padrões de Projeto Utilizados

O sistema CineLog implementa os seguintes padrões de projeto:

Padrão	Local no código	Descrição resumida
Repository	IRepositorios.cs, Repositorios.cs	Abstrai a camada de dados. Hoje: JSON. Amanhã: SQLite/SQL Server sem mudar o Service
Service Layer	CatalogoService.cs	Concentra toda a lógica de negócio em um único ponto
Facade	ConsoleUI.cs	Simplifica a interação com o sistema complexo via console
Dependency Injection	Program.cs	Repositórios injetados no Service via construtor
Value Object	Avaliacao, ListaPersonalizada	Objetos imutáveis definidos pelos seus atributos
DTO (Data Transfer Object)	DTOs/DTOs.cs	Isola o modelo de domínio do contrato JSON exposto pela API, evitando vaziar detalhes internos (como hash de senha)

5. Decisões Iniciais

Linguagem: C#

Framework: ASP.NET Core (API REST)

Frontend: HTML/CSS/JS puro (sem frameworks)

Persistência: JSON via System.Text.Json

Comunicação: HTTP/REST com fetch() (AJAX)

Divisão Inicial criada na primeira parte sendo:

Seção do Trabalho	Responsável(s)
Apresentação do Sistema	Lucas Marinho Blon Rocha e Fábio Sérgio Carvalho Mendonça
Modelagem do Sistema	Guilherme Cândido Vidulino e Eduardo Niquini Santiago
Aplicação dos Quatro Pilares da POO	Matheus Souto dos Santos
Decisões Iniciais	Heitor Freitas Fernandes
Repositório GitHub	Lucas Marinho Blon Rocha e Fábio Sérgio Carvalho Mendonça

6. Repositório GitHub:

<https://github.com/Lucasmarinho12/Catalogo-de-Filmes-POO.git>